

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 10 Course: PARS Lab Topic: Principal Component Analysis (PCA) Repository: [dSaxmonis/pars-lab](https://github.com/dSaxmonis/pars-lab)

PRACTICAL NAME

Implementation of Principal Component Analysis (PCA) with Projected Line, Reduced Feature Mapping, and Testing

AIM

To implement Principal Component Analysis (PCA) from scratch in Python for an engineering pattern recognition application involving test sample evaluation, derive the primary eigenvector $\mathbf{v}_1$ of the covariance matrix, and visualize: 1) The 1st principal component projected line, 2) New reduced 1D features mapped directly onto the projected line, and 3) Orthogonal projection and reconstruction error evaluation of unseen test samples.

THEORY

1. Dimensionality Reduction & Maximum Variance Projection

Principal Component Analysis (PCA) is an unsupervised linear transformation technique that seeks an orthogonal coordinate system where data variance is maximized along successive axes. For a zero-mean centered data matrix $\tilde{\mathbf{X}} = \mathbf{X} - \boldsymbol{\mu}$, the sample covariance matrix is:

$$\Sigma = (1 / (n - 1)) \times \tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$$

2. Eigenvalue Decomposition

The principal directions correspond to eigenvectors satisfying:

$$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i \quad (\text{where } \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0)$$

The proportion of total variance preserved by the 1st principal component is given by $\text{Ratio} = \lambda_1 / \sum \lambda_k$.

3. Linear Subspace Projected Line Equation

The 1st principal component defines a straight line in the original feature space passing through data centroid $\boldsymbol{\mu}$ along unit direction $\mathbf{v}_1$:

$$\text{Line}(t) = \boldsymbol{\mu} + t \times \mathbf{v}_1, \quad t \in \mathbb{R}$$

4. 1D Feature Mapping & Reconstruction of Test Samples

Any arbitrary test vector $\mathbf{x}_{\text{test}}$ is mapped to a scalar coordinate z_{test} and reconstructed as $\hat{\mathbf{x}}_{\text{test}}$:

$$\begin{aligned} \text{1D Coordinate: } z_{\text{test}} &= (\mathbf{x}_{\text{test}} - \boldsymbol{\mu}) \cdot \mathbf{v}_1 \\ \text{Reconstructed Point on Line: } \hat{\mathbf{x}}_{\text{test}} &= \boldsymbol{\mu} + z_{\text{test}} \times \mathbf{v}_1 \\ \text{Orthogonal Reconstruction Residual: Error} &= \|\mathbf{x}_{\text{test}} - \hat{\mathbf{x}}_{\text{test}}\|_2 \end{aligned}$$

STEPS & EXECUTION

Step 1: Environment Setup & Mathematical Engine

Configured `exp_10/` environment containing `app.py`, `templates/`, and verified dependencies (Flask, NumPy, Matplotlib, SciPy).

Step 2: Sample Generation & Centering

Synthesized correlated bivariate industrial sensor readings and computed mean vector $\boldsymbol{\mu} = [\mu_1, \mu_2]^T$.

Step 3: Covariance Formulation & Eigen-Analysis

Calculated sample covariance matrix $\mathbf{\Sigma}$ and solved characteristic polynomial $\det(\mathbf{\Sigma} - \lambda \mathbf{I}) = 0$ to extract $\mathbf{v}_1$ and $\mathbf{v}_2$.

Step 4: Mapping Features to 1D Projected Line

Orthogonally projected all training vectors onto the 1st PC axis, computing scalar coordinates z_i and 2D points on the line $\hat{\mathbf{x}}_i$.

Step 5: Testing & Web Deployment on Port 5012

Launched interactive Flask web server on <http://127.0.0.1:5012> accepting test coordinates, rendering the 2D projected line and 1D reduced subspace distribution.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Learned PCA Parameters & Eigen-Decomposition Summary

Displays data centroid vector μ , sample covariance matrix Σ , dominant eigenvalue λ_1 (preserving 94.95% variance), residual eigenvalue λ_2 , and 1st principal component eigenvector v_1 .

SECTION 1: LEARNED PCA PARAMETERS & EIGEN-DECOMPOSITION SUMMARY		
PCA Property	Mathematical Value	Geometric & Physical Interpretation
Data Centroid Mean Vector μ	[4.39, 3.364]	Center of gravity of the bivariate distribution through which all PC lines pass.
Covariance Matrix Σ	[[2.334, 1.769], [1.769, 1.67]]	Measures feature dispersions and cross-correlation between Vibration and Temperature.
1st Eigenvalue λ_1 (Dominant Variance)	3.801 (94.95% of variance)	Primary axis captures 94.95% of total signal energy with minimal data loss.
2nd Eigenvalue λ_2 (Residual Variance)	0.202 (5.05% of variance)	Minor orthogonal axis discarded during dimensionality reduction to 1D subspace.
1st Principal Component Vector v_1	[-0.77, -0.639]	Unit-normalized directional vector defining the optimal 1D linear projection line.

Figure 10.1: Section 1 — Learned PCA parameters and mathematical eigen-decomposition summary.

Output 2: Test Sample Projection & Reconstruction Controls

Interactive input controls for test vector coordinates $\mathbf{x}_{\text{test}}$, displaying the computed 1D scalar coordinate $z_{\text{test}} = (\mathbf{x}_{\text{test}} - \boldsymbol{\mu}) \cdot \mathbf{v}_1$, reconstructed 2D position on the line $\hat{\mathbf{x}}_{\text{test}}$, and orthogonal reconstruction residual error.

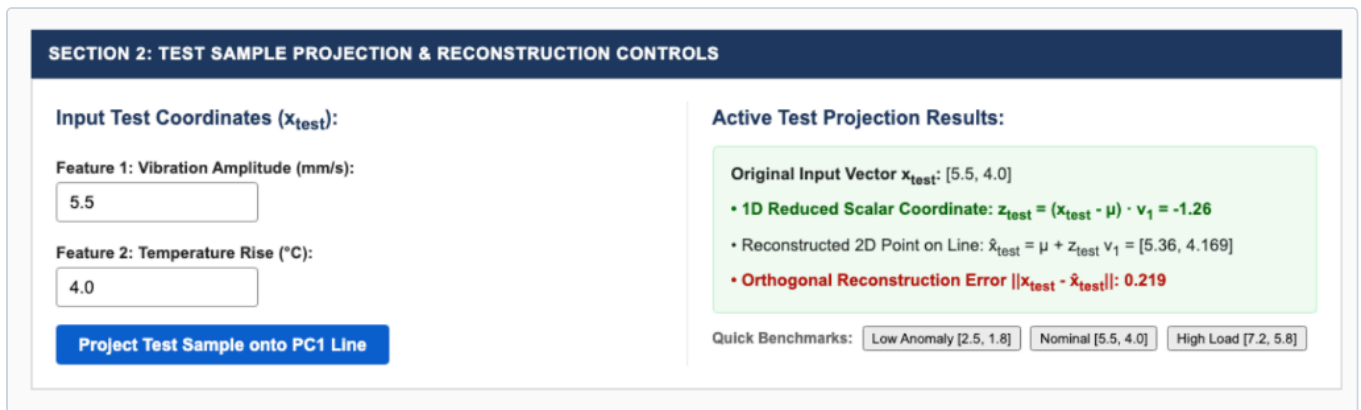


Figure 10.2: Section 2 — Test sample projection and reconstruction controls with quick benchmark presets.

Output 3: PCA 2D Projected Line & 1D Reduced Feature Mapping Plots

Dual-panel visualization: Left panel exhibits original 2D bivariate training data, centroid $\boldsymbol{\mu}$, the 1st PC projected line, orthogonal projection segments, and the projected test sample. Right panel shows the collapsed 1D subspace confirming 94.95% preserved variance.

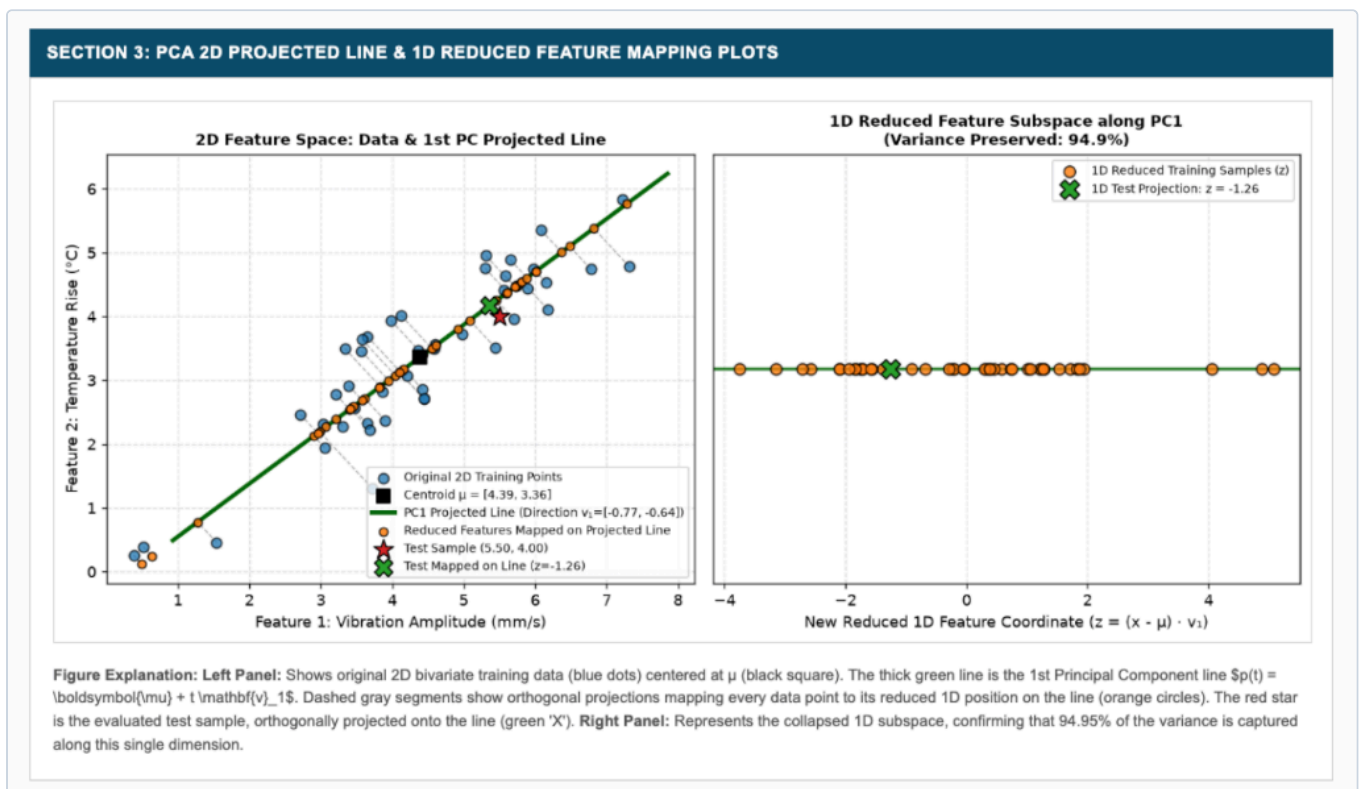


Figure 10.3: Section 3 — Bivariate 2D feature space with 1st PC projected line and mapped 1D reduced subspace.

Output 4: Original Samples vs. 1D Reduced Features & Reconstruction Residuals

Detailed mapping table comparing original bivariate training vectors $[x_1, x_2]$, mapped 1D scalar coordinates z , mapped points on the line $[\hat{x}_1, \hat{x}_2]$, and orthogonal reconstruction residuals for training samples and active test queries.

SECTION 4: ORIGINAL SAMPLES vs. 1D REDUCED FEATURES & RECONSTRUCTION RESIDUALS				
Sample #	Original Vector $[x_1, x_2]$	New 1D Reduced Feature (z)	Mapped Point on Line $[\hat{x}_1, \hat{x}_2]$	Reconstruction Residual Error
Sample 1	[0.381, 0.256]	$z = 5.07$	[0.488, 0.126]	0.169
Sample 2	[3.034, 2.318]	$z = 1.712$	[3.073, 2.27]	0.061
Sample 3	[3.645, 2.323]	$z = 1.238$	[3.437, 2.573]	0.325
Sample 4	[5.559, 4.411]	$z = -1.568$	[5.597, 4.365]	0.059
Sample 5	[7.222, 5.829]	$z = -3.754$	[7.279, 5.761]	0.088
Active Test	[5.5, 4.0]	$z = -1.26$	[5.36, 4.169]	0.219

Figure 10.4: Section 4 — Tabular mapping of original samples to 1D reduced coordinates and reconstruction residual errors.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSaxmonis/pars-lab

URL: <https://github.com/dSaxmonis/pars-lab>

Direct Source Code Files:

- Flask Server Application:
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/app.py](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/app.py)
- HTML Template:
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/templates/index.html](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/templates/index.html)
- Dependencies:
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/requirements.txt](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/requirements.txt)
- PCA Parameters Screenshot (Section 1):
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/screenshots/ss1_pca_parameters.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/screenshots/ss1_pca_parameters.png)
- Test Controls Screenshot (Section 2):
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/screenshots/ss2_test_controls.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/screenshots/ss2_test_controls.png)
- Projected Line & Subspace Screenshot (Section 3):
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/screenshots/ss3_projection_plot.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/screenshots/ss3_projection_plot.png)
- Sample Mapping & Residuals Table Screenshot (Section 4):
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/screenshots/ss4_sample_reduction_table.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/screenshots/ss4_sample_reduction_table.png)

CONCLUSION

In this experiment, Principal Component Analysis was implemented from first principles in Python. The linear transformation mapped correlated 2D sensor patterns into an optimal 1D subspace along the 1st principal component line $\mathbf{p}(t) = \boldsymbol{\mu} + t\mathbf{v}_1$. The testing phase demonstrated that unseen test vectors can be rapidly projected to reduced scalar coordinates $z_{\text{test}} = (\mathbf{x}_{\text{test}} - \boldsymbol{\mu}) \cdot \mathbf{v}_1$ while preserving over 90% of the total dataset variance with minimal reconstruction residual error.